CreaseIQ - project guide

Version 3.1.0 · 18 October 2026

CreaseIQ is an independent educational cricket research application. It is designed for an IPL franchise analyst or scout. It combines match records, clear calculations, a constrained AI workflow and a record of the analyst's reasoning.

Read this guide in order to understand and reproduce the project. Use the chapter selector in the app to find a topic. The same guide is available as PDF and Word files. The complete project ZIP contains the code, processed data, tests and this guide. It never contains the hosting API key or private journal.

1. What the product helps you do

An analyst usually starts with a decision, not a chart. For example: "Which death-bowling players deserve a closer review?" or "Can this squad cover all twenty overs when a leading bowler is unavailable?"

CreaseIQ supports four research tasks:

1. Scouting: rank and compare observed batting or bowling evidence in a selected phase.

2. XI planning: choose eleven supported players and assign a bowler to every over.

3. Player evidence: confirm a player's identity and inspect the selected match sample.

4. Match research: inspect a historical baseline and the results of a chronological model study.

The analyst confirms the context, checks the evidence, adds information from scouting and current team sources, and makes the final decision. A descriptive score is not a forecast, player valuation or guarantee of success.

2. Data coverage, including 2026

The included data is a frozen set of supplied Cricsheet archives. Only the eligible men's matches are included. A match on the cutoff date is excluded; the app uses matches strictly before that date.

Competition	Included matches	Earliest	Latest
IPL	1,242	2008-04-18	2026-05-31
BBL	662	2011-12-16	2026-01-25
Hundred	201	2021-07-22	2026-08-16
T20I	3,592	2005-02-17	2026-10-06
SMAT	695	2016-01-02	2024-12-15

There are 6,392 included matches and 141,377 player-match records. Yes, there is 2026 evidence. This does not mean every 2026 match is present, or that the app updates itself. SMAT ends in 2024. Check the selected date and the last relevant match for each player. Known coverage gaps affect full career totals. For example, the archive has 3,963 T20I runs for Virat Kohli, while the ICC records 4,188. The SMAT 2024/25 sample has only eight matches. Use archive totals as the supplied sample, not an official complete career record. Evaluation & sources contains the detailed accuracy audit and source-linked comparison download.

Archive names, counts and SHA-256 fingerprints are in data/audit.json. The archive hashes identify the exact input files used to build this release. One voided IPL match, duplicate match IDs and other-gender matches are excluded. Super overs are not mixed into the regular innings totals.

The squad file is a supplied 2026 research pool. It can include a withdrawn player and a replacement together. Dated trades and fourteen selected availability notices are applied. Documented outgoing players are blocked after the notice date, even when a capability assumption is supplied. Existing replacements are blocked before their documented join date. Seven documented replacements are missing verified roster metadata; the app names them rather than inventing it. This is partial verification, not a complete current registration list. XI scenarios require a cutoff within 2026. Other seasons need their own verified roster.

The auction CSV contains historical 2013-2022 records. Currency units and transaction types need verification. It is only a manual lookup and is not used to estimate a current price.

3. Run the project on your computer

1. Download the complete project ZIP from Home or Project guide.

2. Extract it. Open a terminal in the extracted creaseiq folder, where app.py is located.

3. Install Python 3.11 or a newer compatible version. Python 3.12 is a suitable choice.

4. Create an isolated environment with python -m venv .venv.

5. On Windows activate it with .venv\Scripts\activate. On macOS or Linux use source .venv/bin/activate.

6. Install the application packages with python -m pip install -r requirements.txt.

7. Run python -m streamlit run app.py.

8. Open the local address printed in the terminal. Start with Home and a guided example.

The processed data is already included. The five large source archives are not needed for normal use. Local evidence works without a paid API. On Windows, start.bat also runs the app after Python and its packages are installed.

4. Connect DeepSeek privately

DeepSeek is optional. Never put a real key in app.py, a public GitHub file, a screenshot or a downloadable project.

For local use, copy .streamlit/secrets.example.toml to .streamlit/secrets.toml. Put your own key in DEEPSEEK_API_KEY in that private file. The real file is ignored by Git. An environment variable named DEEPSEEK_API_KEY is another supported option.

On Streamlit Community Cloud, use the app's Secrets settings and enter the same TOML setting. The repository and source download contain only an empty example. Settings also lets a user supply a temporary session override through a password field. It stays in server memory and is never included in a brief or export.

Each browser session has a limit of six provider calls per UTC day across all franchise desks. The installation has a shared limit of thirty calls. A brief makes at most two calls and may use a cached result for the exact same task and sources. A connection test uses one call. These are small-demo cost controls, not authenticated user accounts or a hard currency limit. Set a provider spending limit too.

Local mode has no API cost. Shared hosting may be free within the host's current limits. DeepSeek usage costs depend on its current input/output token pricing and your provider balance. Check the provider dashboard for actual charges; this project does not promise a fixed monthly bill.

5. Start with the guided examples

The home page explains the workflow: Question, Confirm, Retrieve, Calculate, Check, Review.

Example A - RCB death bowling:

1. Click Load RCB example. It sets RCB, the Scouting task, IPL bowling, Death phase and a 120-ball minimum.

2. Check the cutoff and evidence start date. The default window is the previous three years.

3. Keep Local evidence for the first run, then click Prepare brief.

4. Read raw economy, wickets, legal balls and last relevant match. Compare those values with the descriptive score.

5. Open the evidence trace. Check the source, task and exact filters.

6. Investigate availability, eligibility, video and acquisition before calling a player a recruitment option.

7. Save the brief privately or download its decision pack.

Example B - MI without Jasprit Bumrah:

1. Click Load MI scenario. It sets the XI task and explicitly adds Jasprit Bumrah to Unavailable / exclude.

2. Open Squad & auction scenarios to inspect the remaining roster and assumptions.

3. Review the XI and every over. If the evidence cannot support a legal plan, read the constraint explanation instead of treating a forced answer as useful.

4. Check the alternatives. Force an alternative to compare a counterfactual, then inspect who enters, who leaves and how the schedule changes.

5. Export the complete plan, schedule, constraints and source fingerprints.

Loading an example does not call DeepSeek or automatically run a paid task.

6. Use every workspace

Home explains the purpose, workflow, examples, source coverage and downloads.

Decision desk prepares a checked brief for one of the four tasks. Confirm a player or comparison identity where needed. The free-text question adds purpose; the structured controls define the calculation. Requests for a different competition, discipline, phase, player, season, franchise, matchup or budget need matching controls. Named exclusions must be confirmed. Complete official career claims need a verified complete source; use Player overview for the supplied archive. A question asking for two phases needs separate scoped briefs. Unknown identity searches cannot produce an unrelated broad result. The app does not silently invent missing age, handedness or current price evidence.

Scouting & comparison controls competition, batting/bowling discipline, phase, date window and minimum relevant balls. Search by display name, archive alias or player ID. Compare up to three identities. ALL keeps competitions in separate groups; it does not convert a BBL score into an IPL forecast.

Squad & auction scenarios controls unavailable players, locked players, role minima, optional costs and capability assumptions. Every successful plan includes eleven players and a legal twenty-over assignment. The Alternatives tab supports a forced-player counterfactual. Constraint diagnostics explain unsupported scenarios.

Player dossier starts with Player overview: choose a competition and inspect all supplied phases from its earliest date to before the cutoff. Runs in archive means exactly that sample. Use active research filters switches to the selected research phase and dates. It shows both phase runs and all-phase runs for the same window. The minimum-ball threshold applies to scouting, not dossier visibility. Discipline selects the uncertainty measure; both batting and bowling totals still use the chosen phase. Identical names need an explicit registry choice, with observed teams shown to help. A name such as Rohit Sharma can identify several different people. Historical auction lookup stays manual.

Example: Virat Kohli has 114 IPL Death-phase runs in the 2023-10-18 to before 2026-10-18 window, compared with 2,073 all-phase runs in that same window. The complete supplied IPL overview contains 9,336 runs through 2026-05-31. These are different scopes; do not compare a phase sample with a full career total.

Match research shows the accepted baseline, held-out comparisons and a reproducible forecast export. Confirm Team A, Team B, venue and chasing team. Changing the XI does not change this baseline: player lineups are not model inputs. If the rating snapshot is after the chosen cutoff, current-snapshot estimates are disabled; use dated holdout rows.

Decision journal holds saved briefs and scenarios for this browser session and selected franchise. A fresh session gets a new scope. Another franchise cannot read those records. Export the journal before closing, resetting or redeploying. This is not an authenticated, durable collaboration system.

Evaluation & sources contains the downloadable accuracy audit, official-source numeric checkpoints, current release checks, a rolling historical scouting study, twenty unrated team review tasks, a private review form, data/asset sources and SWOT. A submitted rating is feedback, not proof of professional validation.

Project guide contains this explanation and PDF, Word and source ZIP downloads.

Settings handles temporary credentials, a provider connection test and the current request allowance. Light and Dark appearances and Reduce motion are in the sidebar throughout the app. System reduced-motion preferences are respected too.

7. How the AI workflow works

The research engine performs the calculations. DeepSeek has a deliberately narrow role: selecting useful retrieved fact IDs and approved review actions. It cannot change filters, execute code, select new tools, invent prose facts or set player prices.

1. The analyst chooses the task and confirms the context.

2. The server checks that the question, player, phase, exclusions and any budget agree with those controls.

3. One permitted deterministic tool runs for that task.

4. The server builds fact records from the tool result.

5. In Local mode, those records are rendered directly.

6. In DeepSeek mode, the model receives the question, context and available fact IDs. It returns a small JSON selection. The server accepts only existing IDs and approved actions.

7. An XI must retain all eleven player facts. A player task must retain every requested identity. The full over schedule is rendered by the server.

8. An invalid or unavailable model selection falls back to the checked local facts and is labeled clearly.

9. The brief is tied to the question, full context, method version and source hashes. If any of them changes, the old result is hidden and must be prepared again before saving or exporting.

An evidence check verifies the supported record, task and completeness. It is not a certificate of intelligence, tactical usefulness or future accuracy. Source data and user text are treated as data, not trusted instructions.

8. How match records and phases are calculated

Player IDs come from the match registry. The project registry preserves supplied archive aliases and squad names. A small set of reviewed display corrections makes common names easier to find. It is not a complete copy of the external Cricsheet names register. Unregistered squad rows have distinct planning IDs and cannot borrow another player's match statistics.

A batter's balls faced exclude wides but include deliveries such as no-balls according to the archive's delivery convention. Bowling legal balls exclude wides and no-balls. Bowler runs conceded exclude byes and leg byes. Wickets credited to bowlers exclude run-outs and other non-bowler dismissal types. Dismissals in batting evidence include recorded dismissals according to the processing rules. Inspect match_data.py for the exact inclusion lists.

Strike rate = 100 x runs / balls faced. Economy = 6 x bowler runs conceded / legal balls. Six-ball normalization makes a rate readable for Hundred evidence too; it does not imply equal competition strength.

Powerplay uses the recorded innings powerplay metadata. Missing metadata uses a documented proportional fallback. Death is an analytical final-fifth window of the scheduled innings, after the powerplay. It is not an official field-restriction period. Middle contains the remaining regular deliveries. A reduced innings uses its recorded target or scheduled length. Over notation such as 10.3 means ten over groups plus three balls, not a decimal number of overs. Hundred over groups contain five balls.

Full twenty-over planning uses a fixed 6/10/4 over structure: 36 powerplay, 60 middle and 24 death legal balls. This planning convention is separate from reduced-innings archive phase calculations.

9. Scores and uncertainty in simple terms

Recent records receive more weight. A match 365 days older receives half the weight. Raw totals remain visible and are not reduced by this weighting.

A 120-ball prior pulls small samples toward the selected competition/phase average. Sample weight equals recency-weighted relevant balls divided by that number plus 120. It measures effective evidence volume, not the chance that a player will succeed.

Batting score rewards runs per ball above the scope average and penalizes dismissal rate above that average. The dismissal weight is 18. Bowling score rewards economy better than the scope average and wicket rate above that average. Its wicket weight is 12. These are transparent research choices, not scientifically proven optimal coefficients. creaseiq_core/analytics.py contains the exact formulas.

The XI scenario utility is batting score / 10 plus bowling score. It is descriptive and can be negative. It is not money or expected match wins. The default XI evidence window is three years of IPL records, even when another scouting competition is selected.

Observed-rate intervals resample relevant matches 500 times with a fixed random seed. At least five relevant match samples are needed. The interval describes uncertainty in the observed average under an independent-match assumption. It does not predict future performance or account for all role/opposition changes.

10. XI planning rules and assumptions

The optimizer chooses eleven players and assigns each of twenty overs at the same time. It enforces at most four overseas players, at least one keeper, the active role minima, all locks and exclusions, and an optional analyst budget.

A default supported bowling option needs at least 120 observed IPL legal balls in the three-year window. Phase eligibility additionally needs 36 powerplay, 36 middle or 24 death balls. A supported batting option needs at least 30 observed balls. These are planning thresholds, not official registration rules. Reference Pace/Spin/Unknown tags are curated assumptions.

Each bowler has at most four overs. Every over has one selected bowler. A bowler cannot deliver consecutive overs. The schedule respects each player's total and phase caps. An independent check validates the returned schedule before the UI displays it.

Selection first maximizes the sum of descriptive player utilities. A second solve keeps that XI fixed and arranges its overs using phase-specific descriptive bowling scores. A successful result is optimal within the stated solver tolerance and constraints; it is not claimed to be the best tactical XI in real cricket. A time limit produces an explanation rather than an unproven optimality claim.

Capability assumptions can change a player's whole-over caps and reference style. Each needs a written reason. Unknown is an explicit style choice. An unobserved player also needs an explicit utility assumption. Multiple assumptions can be added, updated and removed. Do not use them to hide absent evidence.

When a budget is active, every available player needs an assumed price. Initial one-crore entries are placeholders. Total XI cost and the full assumption table are exported. They are not live market valuations. This model does not certify Impact Player substitutions, current registration, match-day availability or every competition regulation.

11. Match model and honest evaluation

The research model is a fixed regularized logistic model. It trains on IPL matches before 2023, validates on 2023 and tests on 2024 onward. Ratings update only after each match date, using past results. The pre-toss features are team Elo and recent result form. Post-toss features add chasing status and earlier venue chase outcomes. There are no player-lineup, injury, weather or pitch-report inputs.

Validation has 73 matches and the held-out test has 215. The research post-toss model has test Brier score 0.248577. The training-derived chase-rate baseline has test Brier score 0.247619; lower is better. The research model does not pass the gate requiring a better Brier score in both validation and test. The app therefore displays the baseline. A pre-toss 50% means no validated edge. It is not evidence that team strengths are truly equal.

The baseline chase rate is about 55.07%, estimated from training history. For a confirmed chasing team the app uses that training rate or its complement. Holdout rows include team identities, venue, date, target, method and baseline. Model metrics and independent-outcome Wilson intervals are in the report. Season dependence and limited sample size remain limitations. Do not use a single accuracy percentage as proof of professional forecasting quality.

The historical scouting study uses 24 fixed settings: three follow-up years, two disciplines and four phases. Each ranking uses only the previous three years with at least 60 relevant training balls. It compares the descriptive score with a raw-rate ranking. Top-five players are selected before future exposure is inspected. Follow-up needs 30 balls. Missing future exposure stays counted. Observed-only means have exposure and survivorship bias. The study does not prove scouting superiority or future value.

Twenty team review tasks are included, two per franchise. A reviewer should check evidence, role relevance, clarity and practical usefulness, then write a reason. No authenticated professional ratings have been collected. Automated passes cannot replace that review.

12. Export and reproduce a decision

A CSV adds the team, exclusive cutoff, window start, competition, phase, method version, task and context fingerprint. A decision ZIP adds manifest.json with the full assumptions and SHA-256 hashes of the relevant data and calculation files. XI packs also contain schedule.csv, alternatives.csv and checks.json. Brief packs contain the rendered brief and checked evidence.

To reproduce a decision, keep the pack and the matching project version. Run that project, use the same team and date, copy every filter, lock, exclusion, price and override from the manifest, then compare the result and source hashes. A numerical score can change if the candidate universe or source data changes. Tied optimal schedules may have different equivalent over assignments.

The source ZIP uses an explicit directory/file allowlist. It includes code, assets, processed data, documentation and tests. It excludes .runtime, databases, caches, environment files and real secrets. Private journal exports contain the work deliberately saved by that session; store them appropriately.

13. Rebuild the data and documents

For normal use, the processed data in the project ZIP is enough. To reproduce raw processing, obtain the same archives named in data/audit.json from the original source or your supplied dataset collection. Compare their SHA-256 values. Current downloads may have changed since this snapshot. Check the source's current license and attribution terms before redistribution.

Place sma_json.zip, ipl_json.zip, hnd_json.zip, t20s_json.zip and bbl_json.zip in a separate raw-data folder. Run these commands from the project root:

python match_data.py --source /path/to/raw-data --output data

python prediction.py

python tools/benchmark_scouting.py

The player registry, supplied squad file, dated squad events, roster_status.json availability notices and reference attack types are release inputs too. Preserve their sources and review any changes; rebuilding match aggregates alone does not verify current squads.

Run the tests described in the next chapter. To rebuild PDF and Word documentation, install the optional packages with python -m pip install -r requirements-docs.txt, then run python tools/build_docs.py. These packages are not needed to run the app. Keep USER_GUIDE.md as the source of truth for the downloadable explanation.

14. Test the release

Run python -m unittest discover -s tests -v from the project root. The tests exercise data conventions, identity lookup, supported schedules, scoped task behaviour, model-output rejection, private storage, downloads, startup and the Streamlit interface. Provider fixtures are simulated; they do not spend money or measure live model quality.

Run python tools/compare_official_checkpoints.py --output work/accuracy-audit to recalculate the stored numeric checkpoints against the archive. Official values in that script were transcribed from the linked pages on 18 October 2026; this command does not fetch live records.

The accuracy audit distinguishes complete raw-archive reconciliation from sampled official-source validation. Its comparison CSV keeps coverage gaps and conflicting values visible. A full per-player official certification still requires complete official scorecards and an independently reviewed registry.

For raw reconciliation, run python tools/verify_raw.py --source /path/to/raw-data with the exact archived inputs. This independently recalculates base totals and phase totals from raw deliveries, then compares them with the processed records. The release checks report the actual comparison count and mismatches.

Test all ten franchise desks. For each, run a default XI, exclude its leading scheduled bowler, force an alternative and try a deliberately insufficient budget. Check that successful plans are complete and rejected plans explain the limitation. Then verify player identities, scoped scouting, downloadable packs and historical match exports.

Use the twenty review tasks with a human analyst. Record the task, evidence, ratings and written reasons. Check the deployed app after publishing, including a permitted live provider call, because local tests do not prove that a hosting rebuild or external API connection has succeeded.

15. Deploy on GitHub and Streamlit

1. Create a GitHub repository and upload the project contents so app.py and requirements.txt are at the repository root.

2. Confirm that real secrets, .runtime and .venv are excluded. Commit only the final project and approved data/assets.

3. In Streamlit Community Cloud create an app using that repository, its main branch and app.py.

4. Choose a compatible Python version, such as 3.12, in hosting settings.

5. Add DEEPSEEK_API_KEY to the hosting Secrets setting if you want live selection.

6. Deploy, open Home, run a local example and download the guide.

7. Check all pages and one permitted DeepSeek request. Inspect hosting logs if startup fails. A local pass alone is not a hosted-service check.

Existing Cloud apps normally rebuild when their configured GitHub branch changes. A redeploy may replace local runtime storage, so export private work first. For a real team deployment, add authentication, access control, durable backups and a reviewed data-update process.

16. Architecture and file map

app.py contains the Streamlit pages, controls and display flow. creaseiq_runtime.py loads the complete creaseiq_core source snapshot under its own fingerprinted package name. Identical code is reused on reruns. Changed code and its Streamlit cache keys receive a new release fingerprint. Each loaded module resolves its data and assets relative to the project folder.

creaseiq_core/analytics.py loads linked evidence, computes scouting summaries and solves/validates the XI schedule. agent.py checks task context and optional DeepSeek selection. prediction.py builds chronological research forecasts. evidence.py creates fingerprints and decision manifests. storage.py stores scoped journals, cache and atomic request counts in SQLite. branding.py handles franchise colors and bundled identification marks. downloads.py creates an allowlisted source ZIP.

match_data.py processes raw delivery archives. The root prediction.py is the model-build entry point. tools contains reproducible studies, document generation and raw verification. data contains the processed release inputs and evaluation outputs. assets contains league/team identification marks and source credits. tests contains executable checks. docs contains this guide and its downloadable formats.

The flow is: supplied archive > processed player/match evidence > confirmed task > deterministic calculation > validation > optional DeepSeek fact selection > checked brief > human decision and export.

17. Practical quality boundaries and next research

This is a strong, inspectable capstone foundation. Professional adoption also needs work outside a public demo: a fully reviewed dated player/squad registry, current availability, source rights review, authenticated access, durable team storage, video and scout evidence, broader independent evaluation and monitored live provider behaviour.

A careful next study would collect written analyst ratings for the twenty tasks, evaluate decisions under real role/venue constraints, and test descriptive weights on untouched seasons. Keep these as measured research tasks; do not label them completed without evidence.

Use team colors and logos to identify the selected desk. Restrained transitions support orientation; tables stay stable. Reduced motion is available. Brand marks are used for educational identification and do not imply league or franchise endorsement. Asset credits are in assets/sources.json.

18. Project name and hosting addresses

The product name is CreaseIQ - Franchise Research Studio. Crease refers to the markings on a cricket pitch; IQ stands for intelligence. The name describes using cricket evidence to support informed decisions, with the analyst in control. The source uses creaseiq_core, creaseiq_runtime.py and the optional CREASEIQ_RUNTIME_DIR setting. Existing saved journal records are adopted once when exactly one compatible earlier store is found in the same runtime folder. Export private work before a hosting change. Several earlier stores are left untouched rather than merged.

The public app is https://creaseiq.streamlit.app/ and the repository is https://github.com/br-abishek-07/creaseiq-cricket-capstone. Use docs/HOSTING_HANDOFF.md for deployment checks and data/project_identity.json for project identity.